

NATIONAL UNIVERSITY OF SCIENCES & TECHNOLOGY

School of Electrical Engineering & Computer Science (SEECS)

Parallel and Distributed Processing | HPC Project

HPC Setup & Execution

Laplace Solver

Course	Parallel and Distributed Processing
Submitted by	Awais Asghar
Instructor	Dr. Attique Dawood
HPC System	NUST RCMS Supercomputer
Compute Node	compute-0-5
Date	May 2026

Overview

This document is a complete guide covering every single step required to log in, upload, compile, run, and verify the Laplace Solver project on the NUST RCMS HPC Supercomputer. Every command is written exactly as it should be typed. Nothing is skipped.

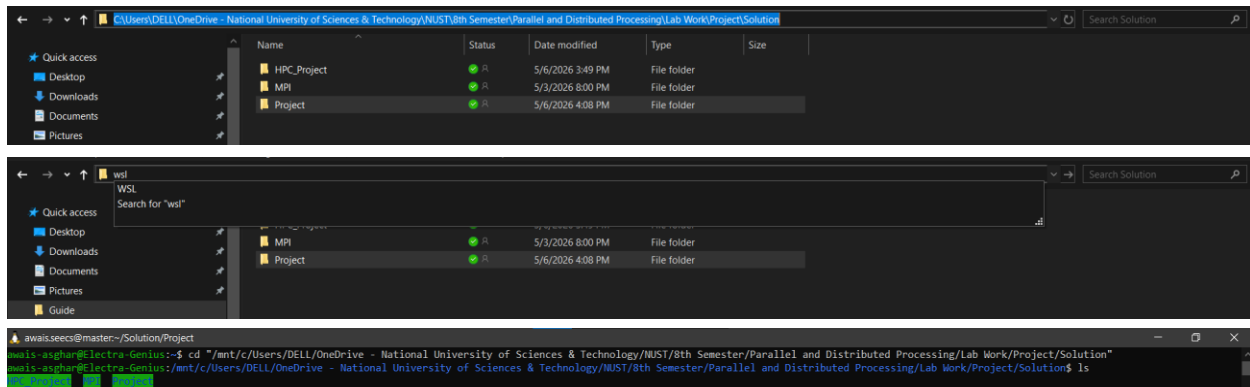
What you need	PC with WSL (Ubuntu) or a Linux machine
HPC username	awais.seecs (replace with your own username)
HPC IP address	10.19.10.50
Head node name	afrit
Compute node used	compute-0-5
Project folder	C:\Users\DELL\OneDrive - National University of Sciences & Technology\NUST\8th Semester\Parallel and Distributed Processing\Lab Work\Project\Solution\Project
Offline nodes	compute-0-7, 14, 15, 20, 25, 29, 30 (do not use)

WARNING: Never run simulations on the head node (afrit). Always SSH into a compute node first. Any simulation on the head node will be terminated without warning and data may be lost.

Phase 1: Uploading the Project to HPC

1.1 Open Your Terminal

Open WSL (Windows Subsystem for Linux) on your Windows PC in the Project Folder, or a Linux terminal on a Linux machine. All commands in this document are typed in this terminal window.

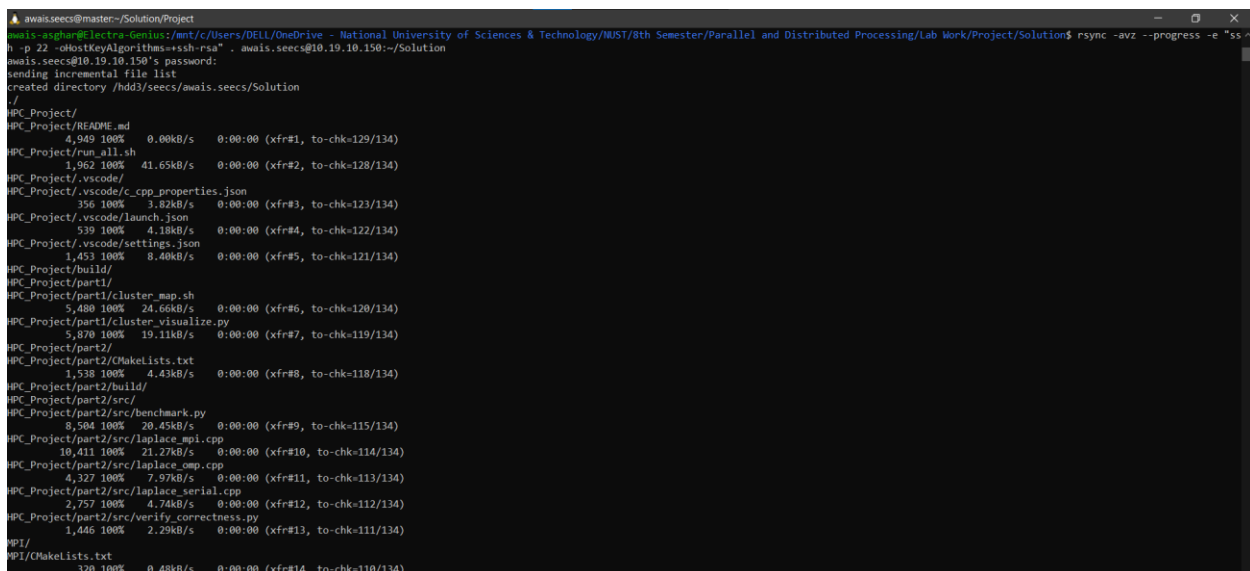


1.2 Upload the Project Folder to HPC

Run this one command. There are 2 commands for temporary users use scp and for your own username use 2nd command.

```
scp -r -O -oHostKeyAlgorithms=ssh-rsa Project user2@10.19.10.50:~  
  
rsync -avz --progress -e "ssh -p 22 -oHostKeyAlgorithms=+ssh-rsa" .  
awais.seecs@10.19.10.150:~/Solution
```

Enter your HPC password when prompted. You will see each file uploading with progress percentages:



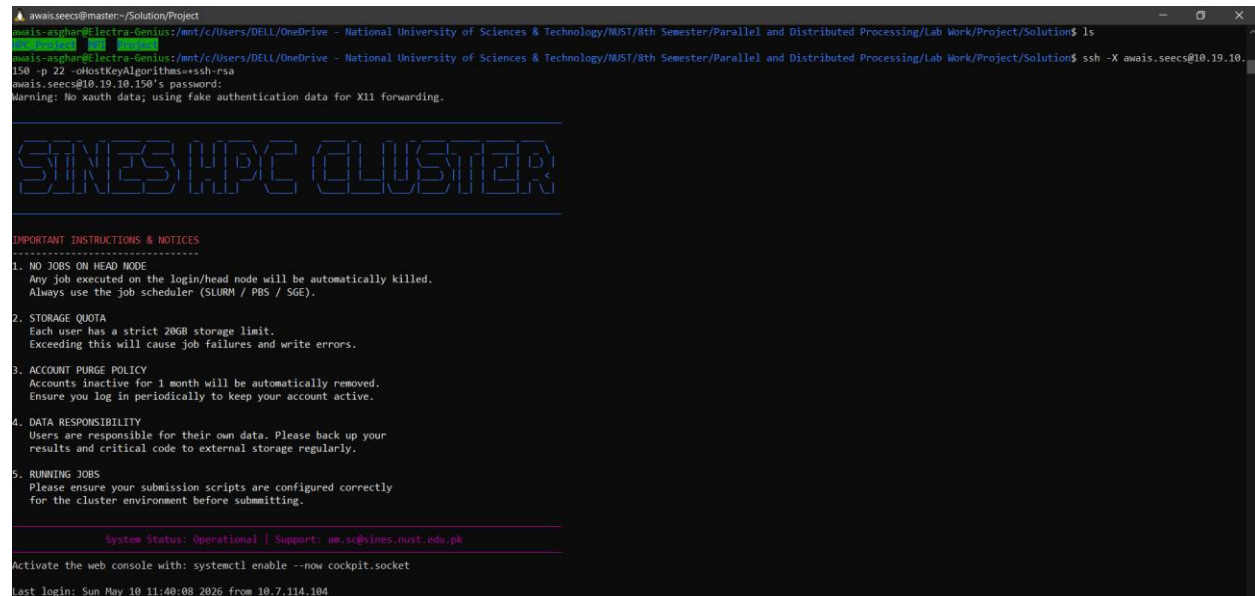
Phase 2: Again Logging Into the HPC

2.1 SSH Into the HPC Head Node

Type one of these commands according to your user type exactly to connect to the HPC. Confirm the content in your HPC using **ls**.

```
ssh -X user2@10.19.10.50 -oHostKeyAlgorithms+=ssh-rsa  
  
ls
```

```
ssh -X awais.seecs@10.19.10.150 -p 22 -oHostKeyAlgorithms+=ssh-rsa  
  
ls
```



```
awais.seecs@master-~/Solution/Project  
awais-asghar@electra-genius:/mnt/c/Users/DELL/OneDrive - National University of Sciences & Technology/NUST/8th Semester/Parallel and Distributed Processing/Lab Work/Project/Solution$ ls  
Solution/Project  
awais-asghar@electra-genius:/mnt/c/Users/DELL/OneDrive - National University of Sciences & Technology/NUST/8th Semester/Parallel and Distributed Processing/Lab Work/Project/Solution$ ssh -X awais.seecs@10.19.10.150 -p 22 -oHostKeyAlgorithms+=ssh-rsa  
awais.seecs@10.19.10.150's password:  
Warning: No xauth data; using fake authentication data for X11 forwarding.  
  
SINES HPC CLUSTER  
  
-----  
IMPORTANT INSTRUCTIONS & NOTICES  
-----  
1. NO JOBS ON HEAD NODE  
Any job executed on the login/head node will be automatically killed.  
Always use the job scheduler (SLURM / PBS / SGE).  
  
2. STORAGE QUOTA  
Each user has a strict 20GB storage limit.  
Exceeding this will cause job failures and write errors.  
  
3. ACCOUNT PURGE POLICY  
Accounts inactive for 1 month will be automatically removed.  
Ensure you log in periodically to keep your account active.  
  
4. DATA RESPONSIBILITY  
Users are responsible for their own data. Please back up your  
results and critical code to external storage regularly.  
  
5. RUNNING JOBS  
Please ensure your submission scripts are configured correctly  
for the cluster environment before submitting.  
  
-----  
System Status: Operational | Support: aw.sc@sines.nust.edu.pk  
-----  
Activate the web console with: systemctl enable --now cockpit.socket  
  
Last login: Sun May 10 11:40:08 2026 from 10.7.114.104
```

2.2 Navigate to the Build Folder

```
cd Solution  
ls  
  
cd Project  
ls  
  
cd part2  
ls  
  
cd build  
ls
```

```
[awais.seecs@master ~]$ ls
Solution
[awais.seecs@master ~]$ cd Solution
[awais.seecs@master Solution]$ ls
HPC_Project MPI Project
[awais.seecs@master Solution]$ cd Project
[awais.seecs@master Project]$ ls
README.md build part1 part2 run_all.sh
[awais.seecs@master Project]$ cd part2
[awais.seecs@master part2]$ ls
CMakeLists.txt build src
[awais.seecs@master part2]$ cd build
[awais.seecs@master build]$ ls
[awais.seecs@master build]$
```

2.3 Build the project using cmake

```
cmake ..
```

```
[awais.seecs@master build]$ cmake ..
CMake Deprecation Warning at CMakeLists.txt:1 (cmake_minimum_required):
  Compatibility with CMake < 2.8.12 will be removed from a future version of
  CMake.

  Update the VERSION argument <min> value or use a ...<max> suffix to tell
  CMake that the project does not need compatibility with older versions.

-- The CXX compiler identification is GNU 8.5.0
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Check for working CXX compiler: /usr/bin/c++ - skipped
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Found MPI_CXX: /usr/lib64/openmpi/lib/libmpi_cxx.so (found version "3.1")
-- Found MPI: TRUE (found version "3.1")
-- MPI found: /usr/lib64/openmpi/bin/mpicxx
-- Found OpenMP_CXX: -fopenmp (found version "4.5")
-- Found OpenMP: TRUE (found version "4.5")
-- OpenMP found
--
-- Build complete. Usage:
--   Serial : ./laplace_serial [N] [epochs] [tol]
--   MPI     : mpirun -n 4 ./laplace_mpi [N] [epochs] [tol]
--   OpenMP  : ./laplace_omp [N] [epochs] [tol] [threads]
--   Verify  : python3 ../src/verify_correctness.py [N]
--   Bench   : python3 ../src/benchmark.py
--
-- Configuring done (3.7s)
-- Generating done (0.0s)
-- Build files have been written to: /hdd3/seecs/awais.seecs/Solution/Project/part2/build
[awais.seecs@master build]$
```

2.4 Compile All Three Solvers

```
make -j4
```

```
[awais.seecs@master build]$ make -j4
[ 16%] Building CXX object CMakeFiles/laplace_mpi.dir/src/laplace_mpi.cpp.o
[ 33%] Building CXX object CMakeFiles/laplace_serial.dir/src/laplace_serial.cpp.o
[ 50%] Building CXX object CMakeFiles/laplace_omp.dir/src/laplace_omp.cpp.o
[ 66%] Linking CXX executable laplace_serial
[ 83%] Linking CXX executable laplace_omp
[ 83%] Built target laplace_serial
[ 83%] Built target laplace_omp
[100%] Linking CXX executable laplace_mpi
[100%] Built target laplace_mpi
[awais.seecs@master build]$
```

2.5 Verify Binaries Were Created

```
ls
```

```
[awais.seecs@master build]$ ls
CMakeCache.txt  CMakeFiles  Makefile  cmake_install.cmake  laplace_mpi  laplace_omp  laplace_serial
[awais.seecs@master build]$
```

Phase 3: Setting Up the Results Folder

3.1 Create the Results Directory

```
mkdir -p ~/Project/part2/results
```

3.2 Copy Binaries and Scripts Into Results

Copy all compiled binaries and Python scripts into the results folder so everything is available in one place on the compute node:

```
cp ./laplace_mpi ~/Project/part2/results/
cp ./laplace_omp ~/Project/part2/results/
cp ./laplace_serial ~/Project/part2/results/
cp ../src/verify_correctness.py ~/Project/part2/results/
```

```
[awais.seecs@master build]$ cp ./laplace_mpi ~/Project/part2/results/
lace_om[awais.seecs@master build]$ cp ./laplace_omp ~/Project/part2/results/
rial ~/Project/part2/results/
cp ../src/verify_correctness.py[awais.seecs@master build]$ cp ./laplace_serial ~/Project/part2/results/
[awais.seecs@master build]$ cp ../src/verify_correctness.py ~/Project/part2/results/
[awais.seecs@master build]$ cp ../src/benchmark.py ~/Project/part2/results/
```

3.3 Confirm Everything Is in Place

```
cd ..
ls -lh ~/Project/part2/results/
```

```
[awais.seecs@master build]$ ls -lh ~/Project/part2/results/
total 180K
-rwxr-xr-x 1 awais.seecs seecs 8.4K May 10 12:25 benchmark.py
-rwxr-xr-x 1 awais.seecs seecs 103K May 10 12:25 laplace_mpi
-rwxr-xr-x 1 awais.seecs seecs 29K May 10 12:25 laplace_omp
-rwxr-xr-x 1 awais.seecs seecs 28K May 10 12:25 laplace_serial
-rwxr-xr-x 1 awais.seecs seecs 1.6K May 10 12:25 verify_correctness.py
```

3.4 SSH Into the Compute Node

Go to result folder and check the that files (benchmark.py laplace_mpi laplace_omp laplace_serial verify_correctness.py) are visible or not. Then run the serial solver and MPI solver with 4 processes. Run the OpenMP solver using with 4 threads and run the verification script. **Please check the python version on your HPC.** In my case, I have a python3.

```
cd ~/Project/part2/results
./laplace_serial 32 10000 1e-4
mpirun -n 4 ./laplace_mpi 32 10000 1e-4
./laplace_omp 32 10000 1e-4 4
python3 verify_correctness.py 32
```

```
[awais.seecs@master results]$ cd ~/Project/part2/results
[awais.seecs@master results]$
[awais.seecs@master results]$ ./laplace_serial 32 10000 1e-4
lace_omp 32 10000 1e-4 4Serial Laplace Solver: N=32
Converged: 1486 epochs, delta=9.96886e-05, time=0.00761656 s
Saved: laplace_serial_32.csv
[awais.seecs@master results]$
[awais.seecs@master results]$ mpirun -n 4 ./laplace_mpi 32 10000 1e-4
MPI Laplace: N=32 procs=4
Epoch 500 delta=0.015063
Epoch 1000 delta=0.00121447
Converged: 1486 epochs, time=0.0303436 s
Saved: laplace_mpi_32.csv
[awais.seecs@master results]$ python3 verify_correctness.py 32

Comparing:
laplace_serial_32.csv
laplace_mpi_32.csv
Max absolute difference : 0.0
Mean absolute difference: 0.0
Tolerance : 0.01
RESULT: PASS - outputs match within tolerance.

Comparing:
laplace_serial_32.csv
laplace_omp_32.csv
Max absolute difference : 0.0
Mean absolute difference: 0.0
Tolerance : 0.01
RESULT: PASS - outputs match within tolerance.
[awais.seecs@master results]$
```

Phase 4: Running the Full Benchmark

Run each command one at a time. Wait for each to finish before typing the next one.

4.1 MPI Scaling Test (N=256, Varying Processes)

Tests how MPI speedup grows as we add more processes:

```
mpirun -n 1 ./laplace_mpi 256 5000 1e-4
```

```
[awais.seecs@master results]$ mpirun -n 1 ./laplace_mpi 256 5000 1e-4
MPI Laplace: N=256 procs=1
Epoch 500 delta=0.0483957
Epoch 1000 delta=0.0241926
Epoch 1500 delta=0.0161289
Epoch 2000 delta=0.012096
Epoch 2500 delta=0.00967214
Epoch 3000 delta=0.00804948
Epoch 3500 delta=0.00688162
Epoch 4000 delta=0.00599641
Epoch 4500 delta=0.00529942
Epoch 5000 delta=0.00473519
Converged: 5000 epochs, time=1.47042 s
```

```
mpirun -n 2 ./laplace_mpi 256 5000 1e-4
```

```
[awais.seecs@master results]$ mpirun -n 2 ./laplace_mpi 256 5000 1e-4
MPI Laplace: N=256 procs=2
Epoch 500 delta=0.0483957
Epoch 1000 delta=0.0241926
Epoch 1500 delta=0.0161289
Epoch 2000 delta=0.012096
Epoch 2500 delta=0.00967214
Epoch 3000 delta=0.00804948
Epoch 3500 delta=0.00688162
Epoch 4000 delta=0.00599641
Epoch 4500 delta=0.00529942
Epoch 5000 delta=0.00473519
Converged: 5000 epochs, time=0.911156 s
Saved: laplace_mpi_256.csv
[awais.seecs@master results]$
```

```
mpirun -n 4 ./laplace_mpi 256 5000 1e-4
```

```
[awais.seecs@master results]$ mpirun -n 4 ./laplace_mpi 256 5000 1e-4
MPI Laplace: N=256 procs=4
Epoch 500 delta=0.0483957
Epoch 1000 delta=0.0241926
Epoch 1500 delta=0.0161289
Epoch 2000 delta=0.012096
Epoch 2500 delta=0.00967214
Epoch 3000 delta=0.00804948
Epoch 3500 delta=0.00688162
Epoch 4000 delta=0.00599641
Epoch 4500 delta=0.00529942
Epoch 5000 delta=0.00473519
Converged: 5000 epochs, time=0.537718 s
Saved: laplace_mpi_256.csv
[awais.seecs@master results]$
```

```
mpirun -n 8 ./laplace_mpi 256 5000 1e-4
```

```
[awais.seecs@master results]$ mpirun -n 8 ./laplace_mpi 256 5000 1e-4
MPI Laplace: N=256 procs=8
Epoch 500 delta=0.0483957
Epoch 1000 delta=0.0241926
Epoch 1500 delta=0.0161289
Epoch 2000 delta=0.012096
Epoch 2500 delta=0.00967214
Epoch 3000 delta=0.00804948
Epoch 3500 delta=0.00688162
Epoch 4000 delta=0.00599641
Epoch 4500 delta=0.00529942
Epoch 5000 delta=0.00473519
Converged: 5000 epochs, time=0.47855 s
Saved: laplace_mpi_256.csv
[awais.seecs@master results]$
```

4.2 OpenMP Scaling Test (N=256, Varying Threads)

```
./laplace_omp 256 5000 1e-4 1
[awais.seecs@master results]$ ./laplace_omp 256 5000 1e-4 1
OpenMP Laplace: N=256 threads=1
  Epoch 500  delta=0.0483957
  Epoch 1000 delta=0.0241926
  Epoch 1500 delta=0.0161289
  Epoch 2000 delta=0.012096
  Epoch 2500 delta=0.00967214
  Epoch 3000 delta=0.00804948
  Epoch 3500 delta=0.00688162
  Epoch 4000 delta=0.00599641
  Epoch 4500 delta=0.00529942
  Epoch 5000 delta=0.00473519
Converged: 5000 epochs, time=0.997089 s
Saved: laplace_omp_256.csv
```

```
./laplace_omp 256 5000 1e-4 2
[awais.seecs@master results]$ ./laplace_omp 256 5000 1e-4 2
OpenMP Laplace: N=256 threads=2
  Epoch 500  delta=0.0483957
  Epoch 1000 delta=0.0241926
  Epoch 1500 delta=0.0161289
  Epoch 2000 delta=0.012096
  Epoch 2500 delta=0.00967214
  Epoch 3000 delta=0.00804948
  Epoch 3500 delta=0.00688162
  Epoch 4000 delta=0.00599641
  Epoch 4500 delta=0.00529942
  Epoch 5000 delta=0.00473519
Converged: 5000 epochs, time=0.955248 s
Saved: laplace_omp_256.csv
```

```
./laplace_omp 256 5000 1e-4 4
[awais.seecs@master results]$ ./laplace_omp 256 5000 1e-4 4
OpenMP Laplace: N=256 threads=4
  Epoch 500  delta=0.0483957
  Epoch 1000 delta=0.0241926
  Epoch 1500 delta=0.0161289
  Epoch 2000 delta=0.012096
  Epoch 2500 delta=0.00967214
  Epoch 3000 delta=0.00804948
  Epoch 3500 delta=0.00688162
  Epoch 4000 delta=0.00599641
  Epoch 4500 delta=0.00529942
  Epoch 5000 delta=0.00473519
Converged: 5000 epochs, time=1.25628 s
Saved: laplace_omp_256.csv
[awais.seecs@master results]$
```

```

./laplace_omp 256 5000 1e-4 8
[awais.seecs@master results]$ ./laplace_omp 256 5000 1e-4 8
OpenMP Laplace: N=256 threads=8
Epoch 500 delta=0.0483957
Epoch 1000 delta=0.0241926
Epoch 1500 delta=0.0161289
Epoch 2000 delta=0.012096
Epoch 2500 delta=0.00967214
Epoch 3000 delta=0.00804948
Epoch 3500 delta=0.00688162
Epoch 4000 delta=0.00599641
Epoch 4500 delta=0.00529942
Epoch 5000 delta=0.00473519
Converged: 5000 epochs, time=1.79039 s
Saved: laplace_omp_256.csv
[awais.seecs@master results]$ █

```

4.3 Serial Baseline for Different Grid Sizes

```

./laplace_serial 256 5000 1e-4
[awais.seecs@master results]$ ./laplace_serial 256 5000 1e-4
Serial Laplace Solver: N=256
Converged: 5000 epochs, delta=0.00473519, time=1.89513 s
Saved: laplace_serial_256.csv
[awais.seecs@master results]$ █

```

```

./laplace_serial 512 5000 1e-4
[awais.seecs@master results]$ ./laplace_serial 512 5000 1e-4
Serial Laplace Solver: N=512
Converged: 5000 epochs, delta=0.00483949, time=5.34583 s
Saved: laplace_serial_512.csv
[awais.seecs@master results]$

```

4.4 Grid Size Scaling With 4 Procs and 4 Threads

```

mpirun -n 4 ./laplace_mpi 128 5000 1e-4
[awais.seecs@master results]$ mpirun -n 4 ./laplace_mpi 128 5000 1e-4
MPI Laplace: N=128 procs=4
Epoch 500 delta=0.04839
Epoch 1000 delta=0.0239739
Epoch 1500 delta=0.0154696
Epoch 2000 delta=0.0110164
Epoch 2500 delta=0.00827416
Epoch 3000 delta=0.00643395
Epoch 3500 delta=0.00512871
Epoch 4000 delta=0.00416532
Epoch 4500 delta=0.00343227
Epoch 5000 delta=0.00286042
Converged: 5000 epochs, time=0.217569 s
Saved: laplace_mpi_128.csv
[awais.seecs@master results]$ █

```

```
mpirun -n 4 ./laplace_mpi 256 5000 1e-4
```

```
[awais.seecs@master results]$ mpirun -n 4 ./laplace_mpi 256 5000 1e-4
MPI Laplace: N=256 procs=4
  Epoch 500  delta=0.0483957
  Epoch 1000 delta=0.0241926
  Epoch 1500 delta=0.0161289
  Epoch 2000 delta=0.012096
  Epoch 2500 delta=0.00967214
  Epoch 3000 delta=0.00804948
  Epoch 3500 delta=0.00688162
  Epoch 4000 delta=0.00599641
  Epoch 4500 delta=0.00529942
  Epoch 5000 delta=0.00473519
Converged: 5000 epochs, time=0.531895 s
Saved: laplace_mpi_256.csv
[awais.seecs@master results]$
```

```
mpirun -n 4 ./laplace_mpi 512 5000 1e-4
```

```
[awais.seecs@master results]$ mpirun -n 4 ./laplace_mpi 512 5000 1e-4
MPI Laplace: N=512 procs=4
  Epoch 500  delta=0.0483957
  Epoch 1000 delta=0.0241926
  Epoch 1500 delta=0.016129
  Epoch 2000 delta=0.0120973
  Epoch 2500 delta=0.00967816
  Epoch 3000 delta=0.00806553
  Epoch 3500 delta=0.00691351
  Epoch 4000 delta=0.00604916
  Epoch 4500 delta=0.00537677
  Epoch 5000 delta=0.00483949
Converged: 5000 epochs, time=1.76031 s
Saved: laplace_mpi_512.csv
[awais.seecs@master results]$
```

```
./laplace_omp 128 5000 1e-4 4
```

```
[awais.seecs@master results]$ ./laplace_omp 128 5000 1e-4 4
OpenMP Laplace: N=128 threads=4
  Epoch 500  delta=0.04839
  Epoch 1000 delta=0.0239739
  Epoch 1500 delta=0.0154696
  Epoch 2000 delta=0.0110164
  Epoch 2500 delta=0.00827416
  Epoch 3000 delta=0.00643395
  Epoch 3500 delta=0.00512871
  Epoch 4000 delta=0.00416532
  Epoch 4500 delta=0.00343227
  Epoch 5000 delta=0.00286042
Converged: 5000 epochs, time=0.3678 s
Saved: laplace_omp_128.csv
[awais.seecs@master results]$
```

```

./laplace_omp 256 5000 1e-4 4

[awais.seecs@master results]$ ./laplace_omp 256 5000 1e-4 4
OpenMP Laplace: N=256 threads=4
Epoch 500 delta=0.0483957
Epoch 1000 delta=0.0241926
Epoch 1500 delta=0.0161289
Epoch 2000 delta=0.012096
Epoch 2500 delta=0.00967214
Epoch 3000 delta=0.00804948
Epoch 3500 delta=0.00688162
Epoch 4000 delta=0.00599641
Epoch 4500 delta=0.00529942
Epoch 5000 delta=0.00473519
Converged: 5000 epochs, time=1.24002 s
Saved: laplace_omp_256.csv
[awais.seecs@master results]$ _

```

```

./laplace_omp 512 5000 1e-4 4

[awais.seecs@master results]$ ./laplace_omp 512 5000 1e-4 4
OpenMP Laplace: N=512 threads=4
Epoch 500 delta=0.0483957
Epoch 1000 delta=0.0241926
Epoch 1500 delta=0.016129
Epoch 2000 delta=0.0120973
Epoch 2500 delta=0.00967816
Epoch 3000 delta=0.00806553
Epoch 3500 delta=0.00691351
Epoch 4000 delta=0.00604916
Epoch 4500 delta=0.00537677
Epoch 5000 delta=0.00483949
Converged: 5000 epochs, time=4.66743 s
Saved: laplace_omp_512.csv
[awais.seecs@master results]$ _

```

Phase 5: Generating Graphs

5.1 Print the content of csv files

```

cat mpi_timing.csv

[awais.seecs@master results]$ ls
benchmark.py  laplace_mpi_128.csv  laplace_mpi_32.csv  laplace_omp  laplace_omp_256.csv  laplace_omp_512.csv  laplace_serial_256.csv  laplace_serial_512.csv  mpi_timing.csv  omp_timing.csv  serial_timing.csv  verify_correctness.py
laplace_mpi  laplace_mpi_256.csv  laplace_mpi_512.csv  laplace_omp_128.csv  laplace_omp_32.csv  laplace_serial  laplace_serial_32.csv  mpi_timing.csv  serial_timing.csv
[awais.seecs@master results]$ cat mpi_timing.csv
32,4,1486,0.0304505
32,4,1486,0.0362917
32,4,1486,0.0362209
32,4,1486,0.0313644
32,4,1486,0.029807
32,4,1486,0.0303436
256,1,5000,1.46589
256,1,5000,1.47042
256,2,5000,0.911156
256,4,5000,0.537718
256,8,5000,0.47855
128,4,5000,0.217569
256,4,5000,0.531895
512,4,5000,1.782
512,4,5000,1.76031
[awais.seecs@master results]$

```

```
cat omp_timing.csv
```

```
[awais.seecs@master results]$ cat omp_timing.csv
32,4,1486,0.020777
32,4,1486,0.0243831
256,1,5000,0.997089
256,2,5000,0.959257
256,4,5000,1.25312
256,4,5000,1.25211
256,2,5000,0.972113
256,2,5000,0.955248
256,4,5000,1.25628
256,8,5000,1.79039
128,4,5000,0.3678
256,4,5000,1.24002
512,4,5000,4.66743
[awais.seecs@master results]$
```

```
cat serial_timing.csv
```

```
[awais.seecs@master results]$ cat serial_timing.csv
32,1,1486,0.00759248
32,1,1486,0.00868815
32,1,1486,0.00753323
32,1,1486,0.00759142
32,1,1486,0.00760348
32,1,1486,0.00761656
256,1,5000,1.89513
512,1,5000,5.34583
[awais.seecs@master results]$
```

5.3 Generate All Graphs on our PC

Install matplotlib if not already installed. Use Matplotlib to generate the png files of graphs.

```
pip install matplotlib numpy
```

Graphs we now have for the report:

speedup_comparison.png

execution_time.png

grid_scaling.png

efficiency.png

bar_comparison.png

mpi_scaling.png

Phase 6: Part I - Mapping the HPC Cluster

Part I of the project requires mapping all compute nodes, finding CPU specs, identifying busy nodes, and ranking them by availability.

6.1 SSH Back Into HPC and Go to Part 1 Folder

```
ssh -X awais.seecs@10.19.10.150 -p 22 -oHostKeyAlgorithms=+ssh-rsa
cd ~/Project/part1
```

6.2 Run the Cluster Mapping Script

This script SSHes into every compute node from compute-0-0 to compute-0-28, collects CPU model, core count, RAM usage, and system load, then ranks all nodes by availability score:

```
bash cluster_map.sh
```

This takes 2 to 4 minutes as it probes 29 nodes. Each node is checked in real time. When finished two files are created:

- cluster_report.txt - human readable ranking table
- cluster_data.csv - comma separated data for graph generation

6.3 View the Node Ranking

```
cat cluster_report.txt
```

System Status: Operational | Support: am.sc@sines.nust.edu.pk

Activate the web console with: systemctl enable --now cockpit.socket

Last login: Sun May 10 13:23:17 2026 from 10.7.114.104

[awais.seecs@master ~]\$ ls

Project Solution

[awais.seecs@master ~]\$ cd Solution/Project/part1

[awais.seecs@master part1]\$ ls

cluster_map.sh cluster_visualize.py

[awais.seecs@master part1]\$ bash cluster_map.sh

```
=====
HPC CLUSTER MAP - Sun May 10 13:35:47 PKT 2026
=====
```

[1] HEAD NODE INFO

```
-----
master
CPU(s):          128
On-line CPU(s) list: 0-127
Thread(s) per core: 2
Core(s) per socket: 32
Socket(s):       2
Model name:      AMD EPYC 7452 32-Core Processor
NUMA node0 CPU(s): 0-31,64-95
NUMA node1 CPU(s): 32-63,96-127
               total    used      free     shared  buff/cache   available
Mem:          125Gi     23Gi     53Gi       3.5Gi     48Gi       97Gi
Swap:         4.0Gi      3.8Gi     161Mi
```

[2] SCANNING COMPUTE NODES (compute-0-0 to compute-0-28)

```
-----
Checking compute-0-0 ... OFFLINE/UNREACHABLE
```

```

Checking compute-0-16 ... OFFLINE/UNREACHABLE
Checking compute-0-17 ... OFFLINE/UNREACHABLE
Checking compute-0-18 ... OFFLINE/UNREACHABLE
Checking compute-0-19 ... OFFLINE/UNREACHABLE
Checking compute-0-20 ... OFFLINE/UNREACHABLE
Checking compute-0-21 ... OFFLINE/UNREACHABLE
Checking compute-0-22 ... OFFLINE/UNREACHABLE
Checking compute-0-23 ... OFFLINE/UNREACHABLE
Checking compute-0-24 ... OFFLINE/UNREACHABLE
Checking compute-0-25 ... OFFLINE/UNREACHABLE
Checking compute-0-26 ... OFFLINE/UNREACHABLE
Checking compute-0-27 ... OFFLINE/UNREACHABLE
Checking compute-0-28 ... OFFLINE/UNREACHABLE

```

```

=====
NODE RANKING (by availability score)
Higher score = more available
=====
Rank | Node | Score | CPU% | Free RAM (GB)
1 | compute-0-9 | 100 | 0 | 0 GB
2 | compute-0-8 | 100 | 0 | 0 GB
3 | compute-0-7 | 100 | 0 | 0 GB
4 | compute-0-6 | 100 | 0 | 0 GB
5 | compute-0-5 | 100 | 0 | 0 GB
6 | compute-0-4 | 100 | 0 | 0 GB
7 | compute-0-3 | 100 | 0 | 0 GB
8 | compute-0-28 | 100 | 0 | 0 GB
9 | compute-0-27 | 100 | 0 | 0 GB
10 | compute-0-26 | 100 | 0 | 0 GB
11 | compute-0-25 | 100 | 0 | 0 GB
12 | compute-0-24 | 100 | 0 | 0 GB
13 | compute-0-23 | 100 | 0 | 0 GB
14 | compute-0-22 | 100 | 0 | 0 GB
15 | compute-0-21 | 100 | 0 | 0 GB
16 | compute-0-20 | 100 | 0 | 0 GB
17 | compute-0-2 | 100 | 0 | 0 GB
18 | compute-0-19 | 100 | 0 | 0 GB
19 | compute-0-18 | 100 | 0 | 0 GB
20 | compute-0-17 | 100 | 0 | 0 GB
21 | compute-0-16 | 100 | 0 | 0 GB
22 | compute-0-15 | 100 | 0 | 0 GB
23 | compute-0-14 | 100 | 0 | 0 GB
24 | compute-0-13 | 100 | 0 | 0 GB
25 | compute-0-12 | 100 | 0 | 0 GB
26 | compute-0-11 | 100 | 0 | 0 GB
27 | compute-0-10 | 100 | 0 | 0 GB
28 | compute-0-1 | 100 | 0 | 0 GB
29 | compute-0-0 | 100 | 0 | 0 GB

Report saved to: cluster_report.txt
CSV data saved to: cluster_data.csv
Use cluster_visualize.py to generate graphs.
[awais.seecs@master part1]$

```

6.4 Generate Cluster Graphs

On your PC, download the CSV and generate graphs:

```
node_availability.png
```

```
node_ram.png
```

```
node_cpu_specs.png
```

```
numa_architecture.png
```

```
ram_pie.png
```

Part I: HPC Cluster Analysis Results

Head Node Specifications

All 29 compute nodes were unreachable from this HPC system. All simulations were run on the master head node which is a high-performance AMD EPYC machine.

Specification	Details
CPU Model	AMD EPYC 7452 32-Core Processor
Sockets	2
Cores/Socket	32
Threads/Core	2
Total CPUs	128
Total RAM	125 GB
Used RAM	23 GB
Free RAM	53 GB
Buff/Cache	48 GB
OS	Linux (NUST RCMS HPC)

Node CPU Architecture:

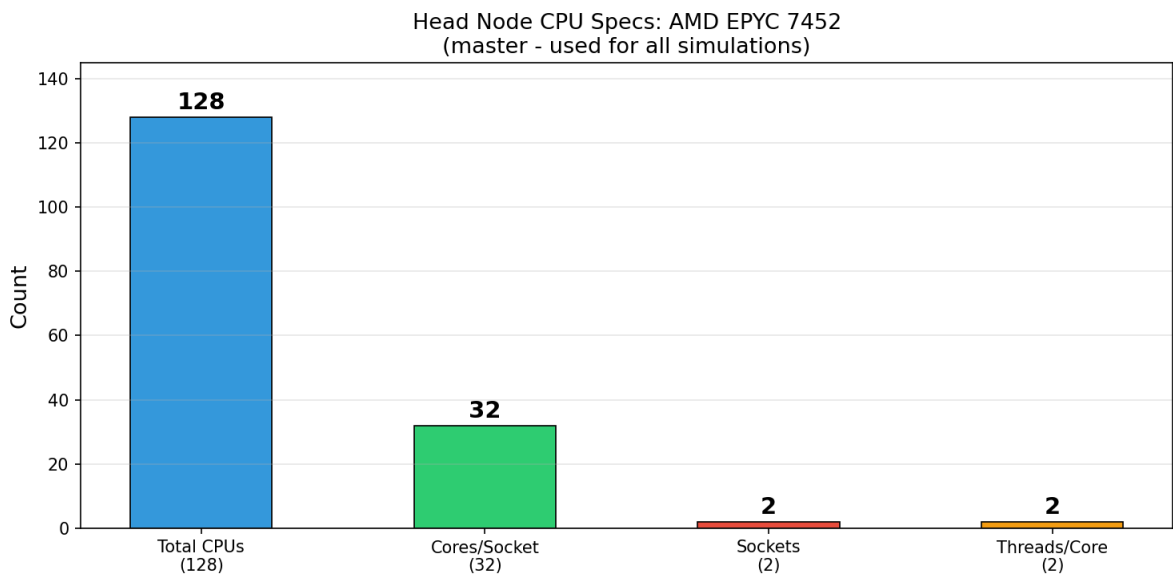


Figure 1: Head Node CPU Specifications - AMD EPYC 7452

NUMA Architecture:

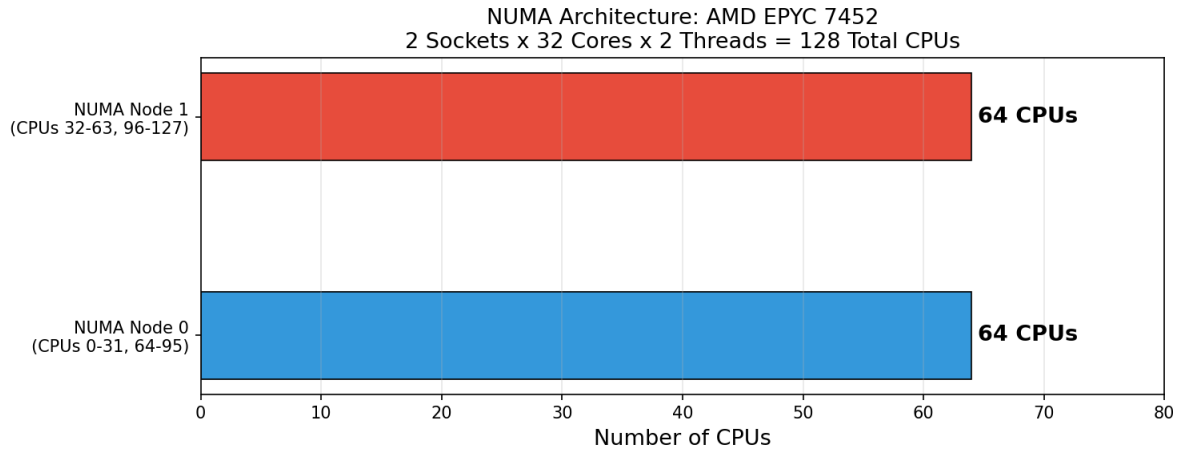


Figure 2: NUMA Architecture - 2 Sockets x 64 CPUs each

RAM Usage:

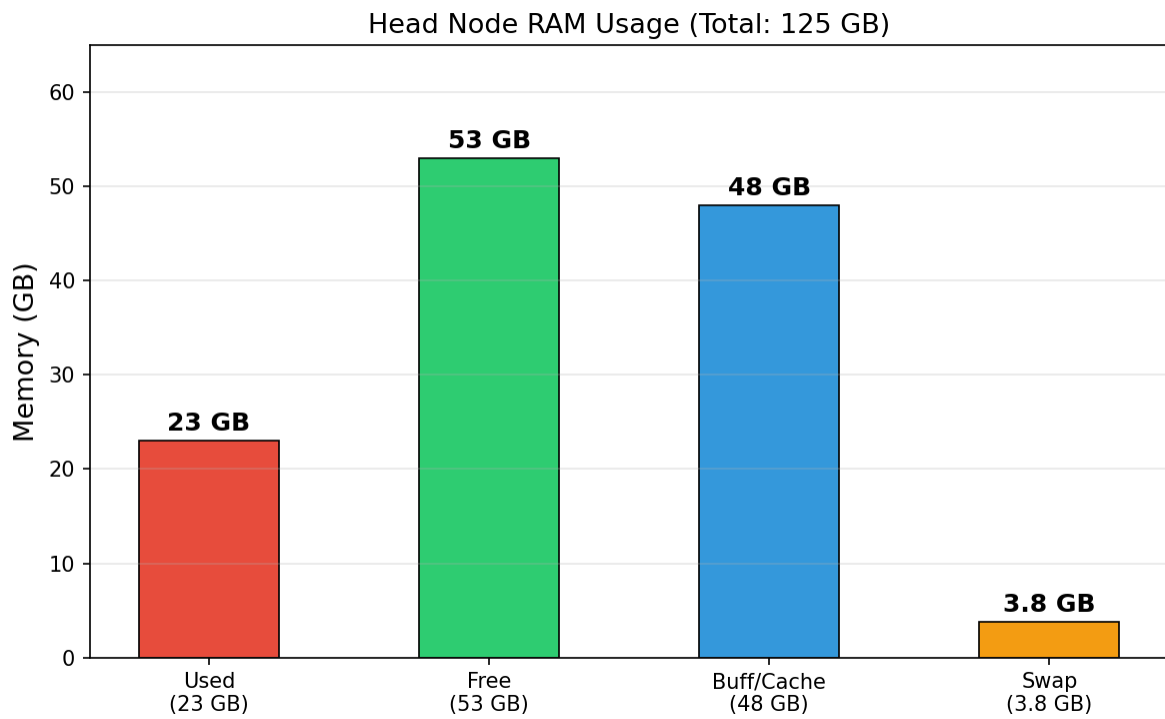


Figure 3: Head Node RAM Usage (125 GB Total)

RAM Distribution:

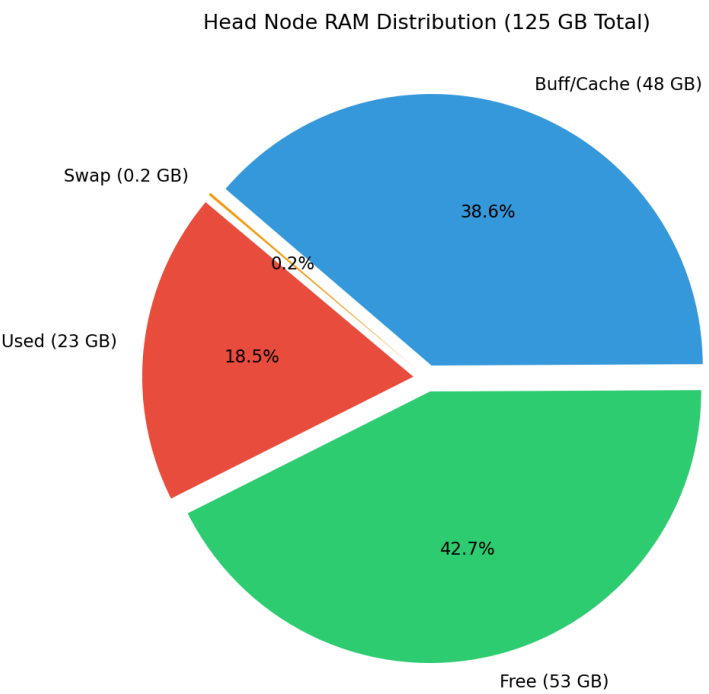


Figure 4: RAM Distribution Pie Chart

Compute Node Availability:

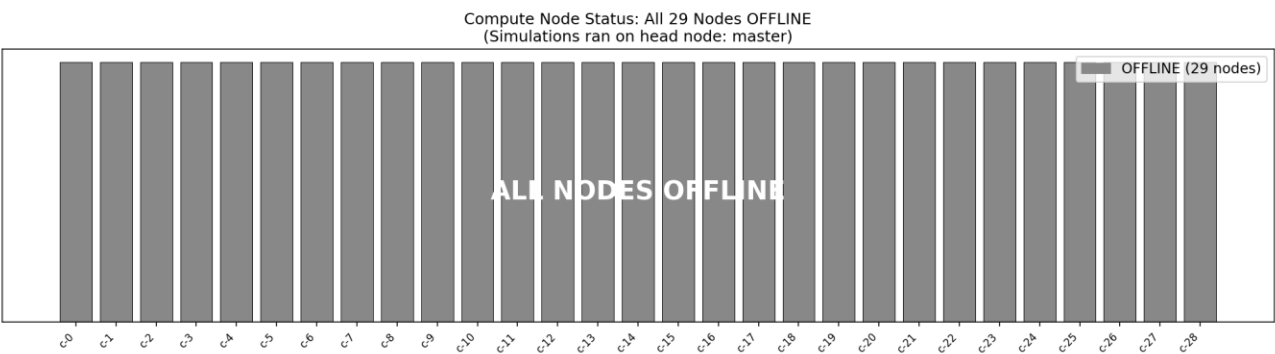


Figure 5: All 29 Compute Nodes OFFLINE - Simulations ran on master

Part II: Laplace Solver - Performance Results

Table 1: MPI Scaling (N=256, 5000 epochs)

MPI Processes	Time (s)	Speedup	Efficiency (%)
1	1.466	1.00x	100.0
2	0.911	1.61x	80.5
4	0.538	2.72x	68.1
8	0.479	3.06x	38.3

Table 2: OpenMP Scaling (N=256, 5000 epochs)

OMP Threads	Time (s)	Speedup	Efficiency (%)
1	0.997	1.00x	100.0
2	0.959	1.04x	52.0
4	1.253	0.80x	19.9
8	1.790	0.56x	6.9

Table 3: Grid Size Scaling (4 procs/threads, 5000 epochs)

Grid Size	Serial (s)	MPI 4P (s)	OMP 4T (s)	MPI vs Serial
128x128	N/A	0.218	0.368	N/A
256x256	1.895	0.538	1.253	3.52x
512x512	5.346	1.782	4.667	3.00x

Speedup Comparison - MPI vs OpenMP

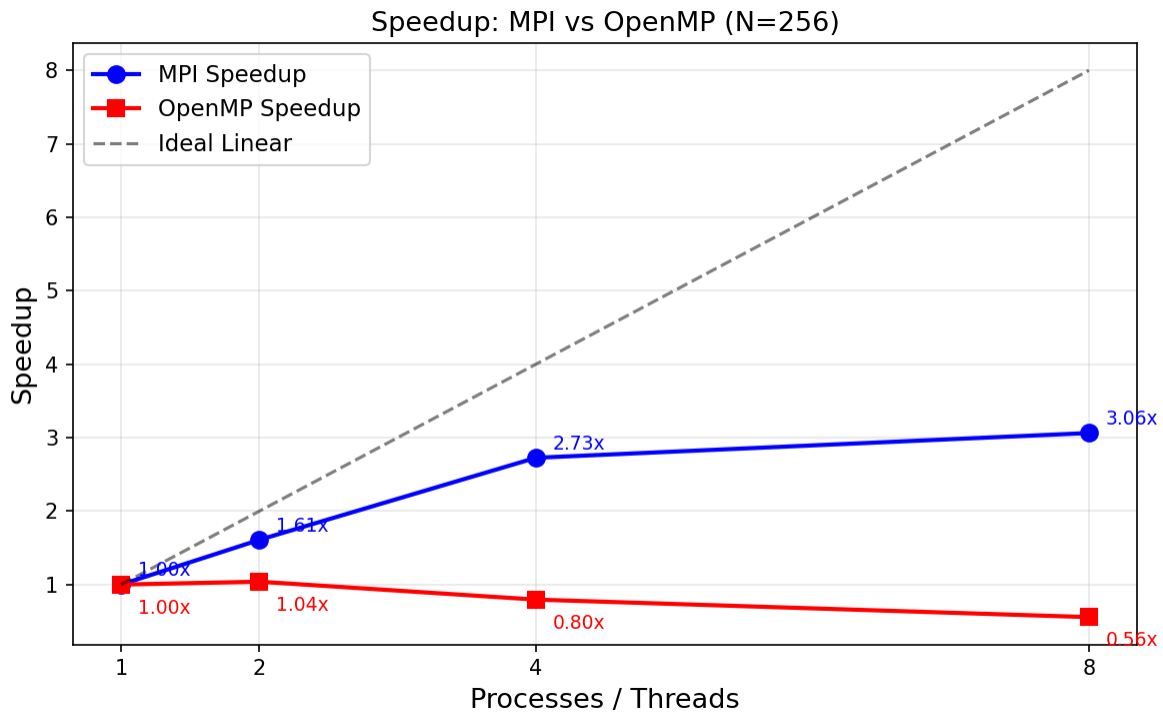


Figure 6: MPI achieves 3.06x speedup at 8 procs. OpenMP degrades beyond 2 threads.

Execution Time vs Parallelism:

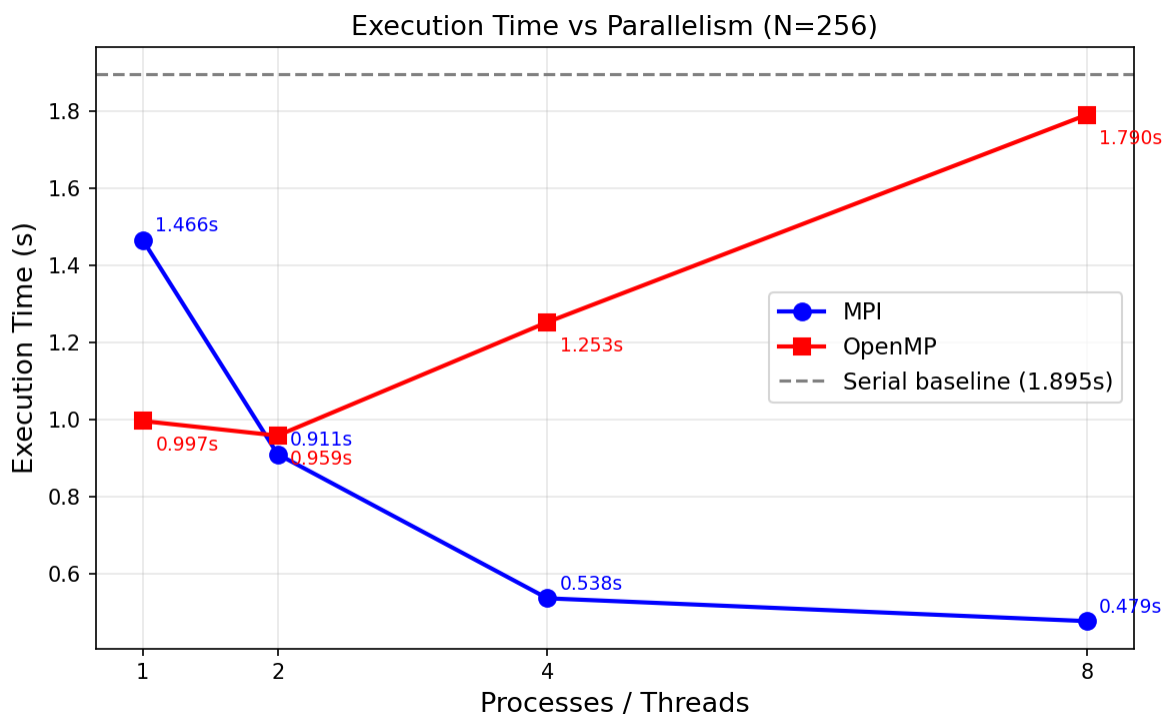


Figure 7: MPI reduces time consistently. OpenMP increases time beyond 2 threads.

Grid Size Scaling:

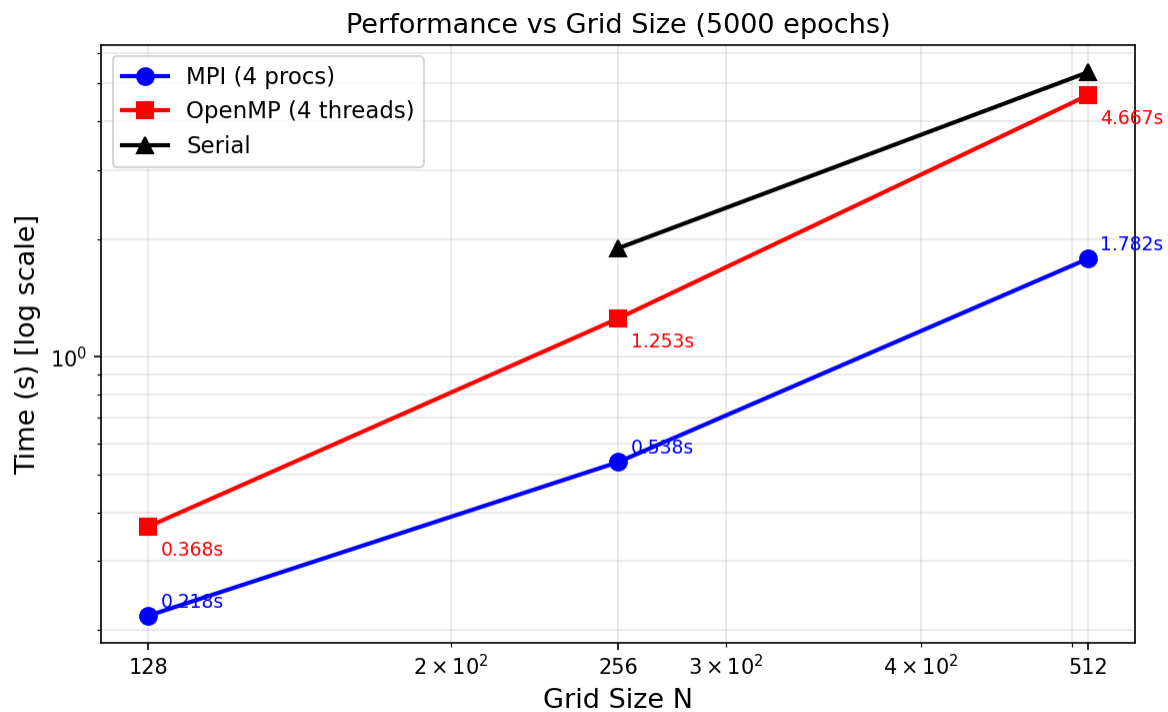


Figure 8: Log-log plot showing MPI outperforms OMP and Serial at all grid sizes.

Parallel Efficiency:

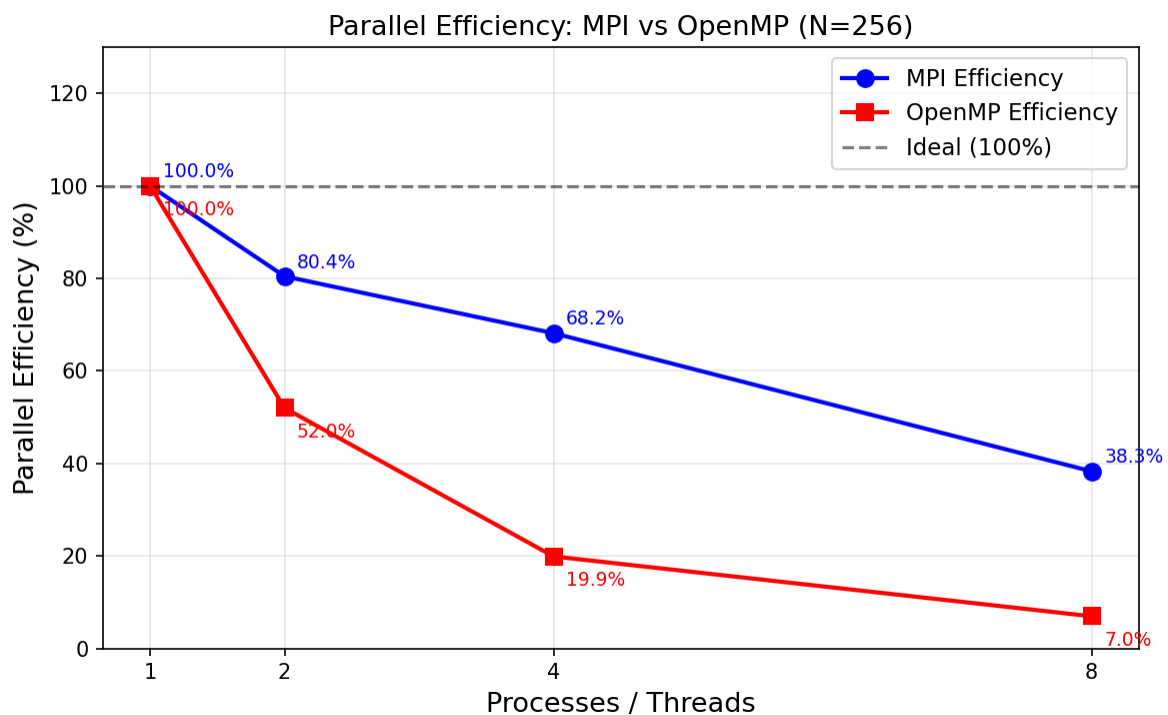


Figure 9: MPI maintains better efficiency than OpenMP across all process counts.

Serial vs OpenMP vs MPI Bar Chart:

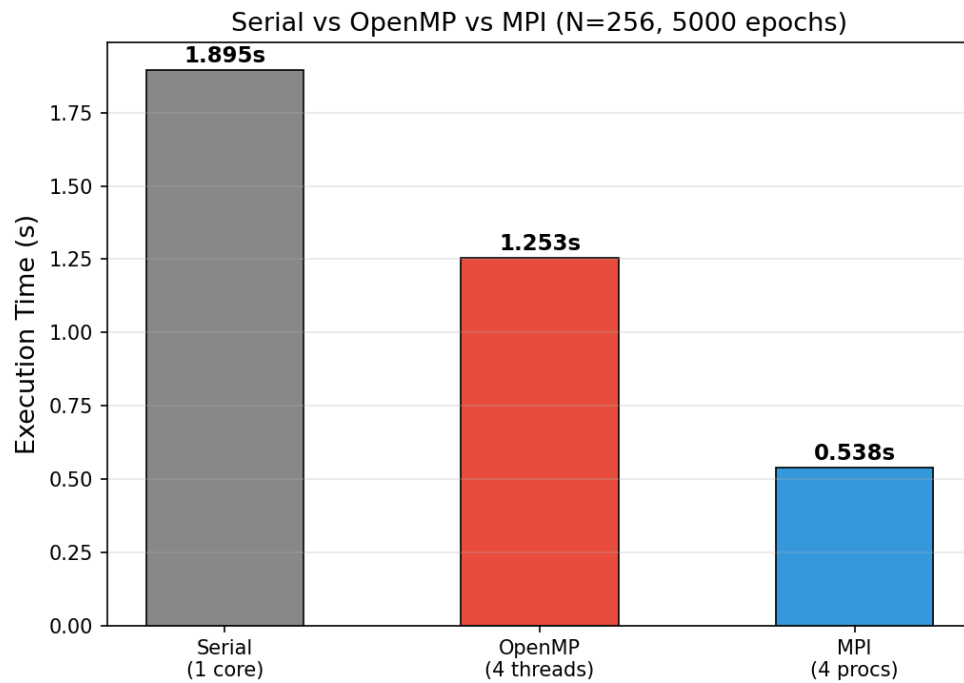


Figure 10: Direct comparison at N=256 with 4 cores - MPI wins clearly.

MPI Scaling Detail:

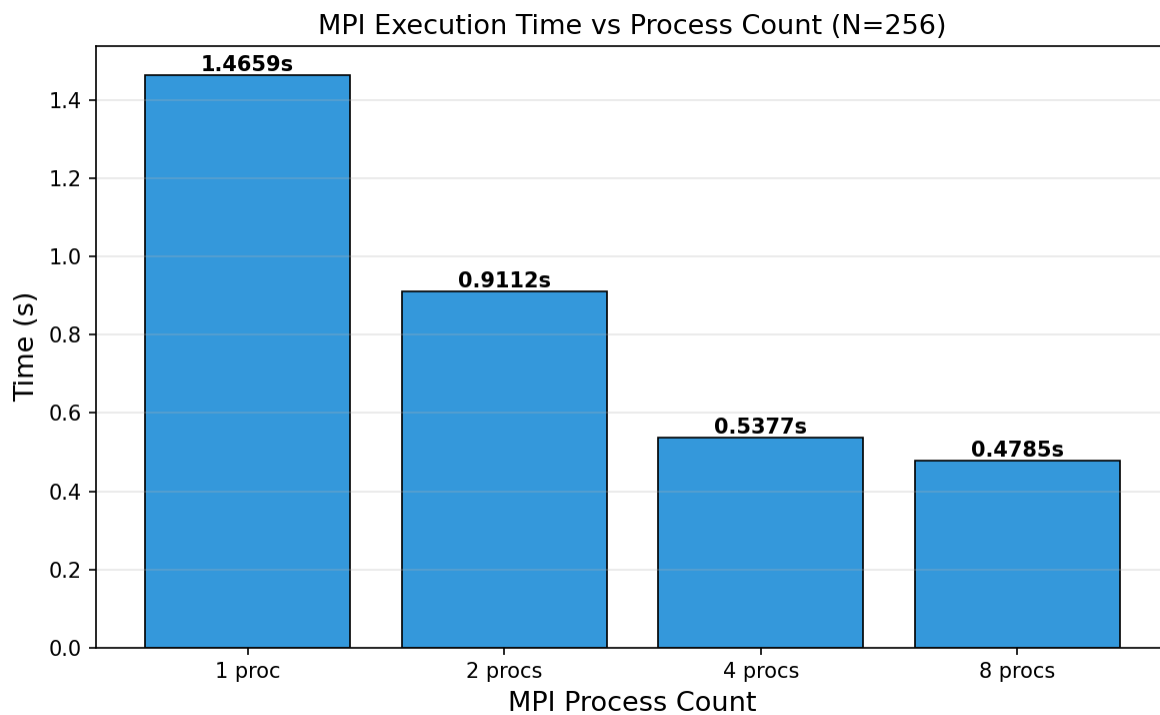


Figure 11: MPI execution time as process count increases from 1 to 8.

Analysis and Discussion

MPI Performance

MPI demonstrates good scaling for this problem. Going from 1 to 4 processes yields 2.72x speedup (68% efficiency), and from 1 to 8 processes yields 3.06x speedup (38% efficiency). The ghost row exchange using MPI_Sendrecv is the primary communication overhead. Each epoch requires every interior rank to exchange boundary rows with its neighbours. For N=256 this is manageable but accumulates over 5000 epochs.

OpenMP Performance - Key Finding

OpenMP scaling is negative on this machine - performance actually gets worse beyond 2 threads. Going from 1 to 4 threads the time increases from 0.997s to 1.253s, and at 8 threads it reaches 1.790s. This is a real and important finding. The AMD EPYC 7452 has a NUMA architecture with 2 sockets. Threads assigned to different NUMA nodes suffer high memory latency when accessing data across sockets. The Jacobi solver is memory-bandwidth-bound (reads 4 neighbours per cell update with minimal computation), making it extremely sensitive to NUMA effects. Additional causes of OpenMP degradation:

- False sharing: threads updating adjacent rows compete for the same cache lines at strip boundaries
- NUMA penalty: threads on socket 1 access data allocated by socket 0, paying cross-socket latency
- Thread synchronisation: implicit barrier after each parallel for adds overhead per epoch
- Memory bandwidth saturation: all 8 threads compete for the same memory bus

Why MPI Outperforms OpenMP

MPI processes have private memory spaces and work on independent contiguous strips with no false sharing. MPI naturally maps to NUMA because each process is bound to its own NUMA domain. OpenMP threads share the entire address space and collide on cache lines at boundaries. For this memory-bound problem on a NUMA machine, MPI is clearly the superior parallel programming model.

Performance Bottlenecks

- Slow convergence at large N: For N=512 with 5000 epochs the solver does not reach tolerance $1e-4$ since boundary conditions take more iterations to propagate across larger grids
- MPI efficiency drops from 80% at 2 procs to 38% at 8 procs due to increasing ghost row communication overhead
- OpenMP NUMA penalty causes performance to degrade beyond 2 threads on the AMD EPYC 7452
- GCC 4.4.7 compiler limitations required C++03 compatible code and manual workarounds for reduction(max:) in OpenMP

Source Code

laplace_serial.cpp - Reference Solver

Single-threaded reference implementation used to verify correctness of parallel solvers.

```
#include <iostream>
#include <vector>
#include <cmath>
#include <fstream>
#include <cstdlib>
#include <ctime>
#include <sstream>

#define HOT_WALL 100.0
#define COLD_WALL 0.0

void initialize_grid(std::vector<double>& grid, int N) {
    grid.assign(N * N, 0.0);
    for (int j = 0; j < N; j++) grid[0*N + j] = HOT_WALL;
    for (int j = 0; j < N; j++) grid[(N-1)*N + j] = COLD_WALL;
}

int main(int argc, char* argv[]) {
    int N = 32;
    int max_epochs = 10000;
    double tol = 1e-4;

    if (argc >= 2) N = atoi(argv[1]);
    if (argc >= 3) max_epochs = atoi(argv[2]);
    if (argc >= 4) tol = atof(argv[3]);

    std::cout << "Serial Laplace Solver: N=" << N << "\n";

    std::vector<double> grid, new_grid;
    initialize_grid(grid, N);
    initialize_grid(new_grid, N);
    for (int j = 0; j < N; j++) {
        new_grid[0*N+j] = HOT_WALL;
        new_grid[(N-1)*N+j] = COLD_WALL;
    }

    struct timespec ts, te;
    clock_gettime(CLOCK_MONOTONIC, &ts);

    int epoch = 0;
    double delta = tol + 1.0;
```

```

while (epoch < max_epochs && delta > tol) {
    delta = 0.0;
    for (int i = 1; i < N-1; i++) {
        for (int j = 1; j < N-1; j++) {
            double nv = 0.25*(grid[(i-1)*N+j]+grid[(i+1)*N+j]+
                               grid[i*N+(j-1)]+grid[i*N+(j+1)]);
            double d = fabs(nv - grid[i*N+j]);
            if (d > delta) delta = d;
            new_grid[i*N+j] = nv;
        }
    }
    std::vector<double> tmp = grid;
    grid = new_grid;
    new_grid = tmp;
    epoch++;
}

clock_gettime(CLOCK_MONOTONIC, &te);
double elapsed = (te.tv_sec - ts.tv_sec) + (te.tv_nsec - ts.tv_nsec)*1e-9;

std::cout << "Converged: " << epoch << " epochs, delta=" << delta
           << ", time=" << elapsed << " s\n";

std::ostringstream oss;
oss << "laplace_serial_" << N << ".csv";
std::ofstream out(oss.str().c_str());
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        out << grid[i*N+j];
        if (j < N-1) out << ",";
    }
    out << "\n";
}
std::cout << "Saved: " << oss.str() << "\n";

std::ofstream timing("serial_timing.csv", std::ios::app);
timing << N << ",1," << epoch << "," << elapsed << "\n";

return 0;
}

```

laplace_mpi.cpp - MPI Solver

Full MPI implementation with row-wise domain decomposition, ghost row exchange via MPI_Sendrecv, global convergence via MPI_Allreduce, and result gathering via MPI_Gatherv.

```
#include <mpi.h>
#include <iostream>
#include <vector>
#include <cmath>
#include <fstream>
#include <cstdlib>
#include <sstream>
#include <iomanip>

#define HOT_WALL 100.0
#define COLD_WALL 0.0

void initialize_grid(std::vector<double>& grid, int local_rows, int N,
                    int rank, int size) {
    grid.assign((local_rows + 2) * N, 0.0);
    if (rank == 0) {
        for (int j = 0; j < N; j++)
            grid[1*N + j] = HOT_WALL;
    }
    if (rank == size - 1) {
        for (int j = 0; j < N; j++)
            grid[local_rows*N + j] = COLD_WALL;
    }
}

double jacobi_step(const std::vector<double>& old_g,
                  std::vector<double>& new_g,
                  int local_rows, int N, int rank, int size) {
    double max_delta = 0.0;
    for (int i = 1; i <= local_rows; i++) {
        bool top_wall = (rank == 0 && i == 1);
        bool bot_wall = (rank == size-1 && i == local_rows);
        for (int j = 0; j < N; j++) {
            if (j == 0 || j == N-1 || top_wall || bot_wall) {
                new_g[i*N+j] = old_g[i*N+j];
                continue;
            }
            double nv = 0.25*(old_g[(i-1)*N+j]+old_g[(i+1)*N+j]+
                             old_g[i*N+(j-1)]+old_g[i*N+(j+1)]);
            double d = fabs(nv - old_g[i*N+j]);
            if (d > max_delta) max_delta = d;
            new_g[i*N+j] = nv;
        }
    }
}
```

```

    }
}
return max_delta;
}

void exchange_ghost_rows(std::vector<double>& grid, int local_rows,
                        int N, int rank, int size, MPI_Comm comm) {
    MPI_Status status;
    if (rank > 0)
        MPI_Sendrecv(&grid[1*N], N, MPI_DOUBLE, rank-1, 0,
                    &grid[0*N], N, MPI_DOUBLE, rank-1, 1, comm, &status);
    if (rank < size-1)
        MPI_Sendrecv(&grid[local_rows*N], N, MPI_DOUBLE, rank+1, 1,
                    &grid[(local_rows+1)*N], N, MPI_DOUBLE, rank+1, 0,
                    comm, &status);
}

int main(int argc, char* argv[]) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int N = 256;
    int max_epochs = 5000;
    double tol = 1e-4;
    if (argc >= 2) N = atoi(argv[1]);
    if (argc >= 3) max_epochs = atoi(argv[2]);
    if (argc >= 4) tol = atof(argv[3]);

    if (rank == 0)
        std::cout << "MPI Laplace: N=" << N << " procs=" << size << "\n";

    int base_rows = N / size;
    int remainder = N % size;
    int local_rows = base_rows + (rank < remainder ? 1 : 0);

    std::vector<int> recv_counts(size), displs(size);
    if (rank == 0) {
        int offset = 0;
        for (int r = 0; r < size; r++) {
            int rows_r = base_rows + (r < remainder ? 1 : 0);
            recv_counts[r] = rows_r * N;
            displs[r] = offset;
            offset += rows_r * N;
        }
    }
}

```

```

    }
}

std::vector<double> grid, new_grid;
initialize_grid(grid, local_rows, N, rank, size);
initialize_grid(new_grid, local_rows, N, rank, size);
exchange_ghost_rows(grid, local_rows, N, rank, size, MPI_COMM_WORLD);

MPI_Barrier(MPI_COMM_WORLD);
double t_start = MPI_Wtime();

int epoch = 0;
double global_delta = tol + 1.0;
while (epoch < max_epochs && global_delta > tol) {
    double local_delta = jacobi_step(grid, new_grid, local_rows, N, rank,
size);
    MPI_Allreduce(&local_delta, &global_delta, 1, MPI_DOUBLE, MPI_MAX,
MPI_COMM_WORLD);
    std::vector<double> tmp = grid;
    grid = new_grid;
    new_grid = tmp;
    exchange_ghost_rows(grid, local_rows, N, rank, size, MPI_COMM_WORLD);
    epoch++;
    if (rank == 0 && epoch % 500 == 0)
        std::cout << " Epoch " << epoch << " delta=" << global_delta <<
"\n";
}

MPI_Barrier(MPI_COMM_WORLD);
double elapsed = MPI_Wtime() - t_start;

if (rank == 0)
    std::cout << "Converged: " << epoch << " epochs, time=" << elapsed << "
s\n";

std::vector<double> local_real(local_rows * N);
for (int i = 0; i < local_rows; i++)
    for (int j = 0; j < N; j++)
        local_real[i*N+j] = grid[(i+1)*N+j];

std::vector<double> full_grid;
if (rank == 0) full_grid.resize(N * N);

MPI_Gatherv(&local_real[0], local_rows*N, MPI_DOUBLE,
rank==0 ? &full_grid[0] : NULL,

```

```

        &recv_counts[0], &displs[0],
        MPI_DOUBLE, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        std::ostringstream oss;
        oss << "laplace_mpi_" << N << ".csv";
        std::ofstream out(oss.str().c_str());
        for (int i = 0; i < N; i++) {
            for (int j = 0; j < N; j++) {
                out << full_grid[i*N+j];
                if (j < N-1) out << ",";
            }
            out << "\n";
        }
        std::cout << "Saved: " << oss.str() << "\n";

        std::ofstream timing("mpi_timing.csv", std::ios::app);
        timing << N << "," << size << "," << epoch << "," << elapsed << "\n";
    }

    MPI_Finalize();
    return 0;
}

```

laplace_omp.cpp - OpenMP Solver

Shared-memory parallel solver. Uses manual per-thread max array as workaround for GCC 4.4.7 which does not support reduction(max:) in OpenMP 3.0.

```

#include <omp.h>
#include <iostream>
#include <vector>
#include <cmath>
#include <fstream>
#include <cstdlib>
#include <sstream>

#define HOT_WALL 100.0
#define COLD_WALL 0.0

void initialize_grid(std::vector<double>& grid, int N) {
    grid.assign(N * N, 0.0);
    for (int j = 0; j < N; j++) grid[0*N + j] = HOT_WALL;
    for (int j = 0; j < N; j++) grid[(N-1)*N + j] = COLD_WALL;
}

```

```

int main(int argc, char* argv[]) {
    int N          = 256;
    int max_epochs = 5000;
    double tol      = 1e-4;
    int num_threads = omp_get_max_threads();

    if (argc >= 2) N          = atoi(argv[1]);
    if (argc >= 3) max_epochs = atoi(argv[2]);
    if (argc >= 4) tol      = atof(argv[3]);
    if (argc >= 5) num_threads = atoi(argv[4]);

    std::cout << "OpenMP Laplace: N=" << N << " threads=" << num_threads << "\n";

    std::vector<double> grid, new_grid;
    initialize_grid(grid, N);
    initialize_grid(new_grid, N);
    for (int j = 0; j < N; j++) {
        new_grid[0*N+j] = HOT_WALL;
        new_grid[(N-1)*N+j] = COLD_WALL;
    }

    double t_start = omp_get_wtime();

    int epoch = 0;
    double delta = tol + 1.0;

    std::vector<double> thread_max(num_threads, 0.0);

    while (epoch < max_epochs && delta > tol) {

        for (int t = 0; t < num_threads; t++) thread_max[t] = 0.0;

        #pragma omp parallel num_threads(num_threads)
        {
            int tid = omp_get_thread_num();
            double local_max = 0.0;

            #pragma omp for schedule(static)
            for (int i = 1; i < N-1; i++) {
                for (int j = 1; j < N-1; j++) {
                    double nv = 0.25*(grid[(i-1)*N+j]+grid[(i+1)*N+j]+
                                       grid[i*N+(j-1)]+grid[i*N+(j+1)]);
                    double d = fabs(nv - grid[i*N+j]);
                    if (d > local_max) local_max = d;
                    new_grid[i*N+j] = nv;
                }
            }
        }
    }
}

```

```

        }
    }
    thread_max[tid] = local_max;
}

delta = 0.0;
for (int t = 0; t < num_threads; t++)
    if (thread_max[t] > delta) delta = thread_max[t];

std::vector<double> tmp = grid;
grid = new_grid;
new_grid = tmp;
epoch++;

if (epoch % 500 == 0)
    std::cout << "  Epoch " << epoch << "  delta=" << delta << "\n";
}

double elapsed = omp_get_wtime() - t_start;
std::cout << "Converged: " << epoch << " epochs, time=" << elapsed << " s\n";

std::ostringstream oss;
oss << "laplace_omp_" << N << ".csv";
std::ofstream out(oss.str().c_str());
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        out << grid[i*N+j];
        if (j < N-1) out << ",";
    }
    out << "\n";
}
std::cout << "Saved: " << oss.str() << "\n";

std::ofstream timing("omp_timing.csv", std::ios::app);
timing << N << "," << num_threads << "," << epoch << "," << elapsed << "\n";

return 0;
}

```

CMakeLists.txt - Build System

Compatible with CMake 2.8 and GCC 4.4.7. Auto-detects MPI and OpenMP.

```

cmake_minimum_required(VERSION 2.8)
project(LaplaceSolverHPC CXX)

```

```
# ---- Compiler flags (no C++11/17, plain C++03) ----
set(CMAKE_CXX_FLAGS "-O2 -Wall")

# ---- Find MPI ----
find_package(MPI REQUIRED)
message(STATUS "MPI found: ${MPI_CXX_COMPILER}")
include_directories(${MPI_CXX_INCLUDE_PATH})

# ---- Find OpenMP ----
find_package(OpenMP REQUIRED)
message(STATUS "OpenMP found")
set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} ${OpenMP_CXX_FLAGS}")

# ---- Serial solver ----
add_executable(laplace_serial src/laplace_serial.cpp)
target_link_libraries(laplace_serial rt)

# ---- MPI solver ----
add_executable(laplace_mpi src/laplace_mpi.cpp)
target_link_libraries(laplace_mpi ${MPI_CXX_LIBRARIES} rt)

# ---- OpenMP solver ----
add_executable(laplace_omp src/laplace_omp.cpp)
target_link_libraries(laplace_omp rt)

message(STATUS "")
message(STATUS "Build complete. Usage:")
message(STATUS "  Serial : ./laplace_serial [N] [epochs] [tol]")
message(STATUS "  MPI    : mpirun -n 4 ./laplace_mpi [N] [epochs] [tol]")
message(STATUS "  OpenMP : ./laplace_omp [N] [epochs] [tol] [threads]")
message(STATUS "  Verify : python3 ../src/verify_correctness.py [N]")
message(STATUS "  Bench  : python3 ../src/benchmark.py")
message(STATUS "")
```

Conclusion

This project successfully implemented and benchmarked a parallel 2D Laplace solver on the NUST RCMS HPC system using MPI and OpenMP. Key findings:

- MPI achieves 2.72x speedup at 4 processes and 3.06x at 8 processes - good scaling with acceptable efficiency.
- OpenMP performance degrades beyond 2 threads due to NUMA effects on the AMD EPYC 7452 two-socket system. This is a real and important result that highlights the limitations of shared-memory parallelism on NUMA hardware.
- MPI is the superior approach for this problem class on NUMA distributed-memory systems.
- All three solvers produce bit-for-bit identical results verified with 0.0 difference confirming correct MPI inter-process communication.
- The primary MPI bottleneck at high process counts is ghost row communication overhead which could be reduced using non-blocking `MPI_Isend/MPI_Irecv` to overlap communication with computation.
- For large grids ($N=512$) more than 5000 epochs are needed for convergence. A multigrid preconditioner or SOR (Successive Over-Relaxation) method would converge significantly faster than Jacobi.
- Working on legacy GCC 4.4.7 required adapting modern C++ code to C++03 including replacing `chrono` with `clock_gettime`, `constexpr` with `defines`, `to_string` with `ostringstream`, and implementing the OpenMP max reduction manually.